

Pruning appliqué à l'échantillonnage de signal sur graphe

Stage 2A de Maëlle Gabens, encadrée par Bastien Pasdeloup et Yassine El Ouahidi

Objectif : appliquer la méthode de pruning développée par Yassine et exposée dans “*Pruning Graph Convolutional Networks to select meaningful graph fréquences for fMRI decoding*” pour faire de la sélection de noeuds sur un graphe et reconstruire le signal samplé.

Abstract : Le but est de déterminer quels sont les nœuds les plus intéressants pour un signal donné se propageant sur un graphe donné, en utilisant le pruning pour améliorer le choix des éléments à garder. Plusieurs Multi-Layer Perceptron ont été mis en place pour voir les différences de performance en fonction de l'endroit si le pruning était appliqué et de comment il était appliqué. Les meilleurs résultats sont obtenus lorsque le graphe est injecté dans le modèle (Net2), ce qui revient à diffuser le signal échantillonné. Ainsi, la reconstruction du signal est prise en compte dans le choix des nœuds à garder. La base de données utilisée pour tester les programmes est MNIST. Les résultats quand on conserve $k=100$ noeuds sont de 93.62% pour l'accuracy et de 0.0137 pour la loss ce qui est inférieur au résultats obtenus lorsqu'on échantillonne pas le signal : 97,11 % pour l'accuracy et 0.005 pour la loss.

Table des matières :

Dossier	2
Pruning	2
Paramètres	2
Fonction train	2
Les modèles	3
Les modèles aléatoires	3
Les modèles utilisant le pruning	3
Les modèles utilisant pruning et diffusion	4
Table de récap	4
Résultats	4
Hyperparamètres	4
Table des résultats	5
Conclusion et ouverture	7

I. Dossier

Le dossier comporte ce **README**, le notebook commenté **main.ipynb** , le fichier **main.py** correspondant, un dossier **Resultats** contenant une table des résultats **results.csv** et un réseau entraîné **MLPclass.joblib** qui sert à calculer l'accuracy des modèles.

II. Pruning

Le pruning consiste à obliger les poids auxquels il est appliqué de tendre graduellement vers 0. Pour ce faire, un terme est ajouté au gradient des poids. Ce terme se calcule de la manière suivante : le poid * . Le but du select weight decay (swd) est d'être très petit au début pour ne pas influencer sur le modèle. Puis qu'il devienne très grand sur la fin de l'entraînement pour pousser les poids à tendre vers 0.

III. Paramètres

Base de données :

MNIST qui contient 70 000 images de chiffres manuscrits de 28*28 pixels, pour entraîner le modèle. Les données ont été séparées en train (50 000), validation (10 000) et test (10 000). Les tailles des batchs sont de 2 000 pour l'entraînement et le test et de 1 000 pour la validation.

Construction du graphe : Prendre les 4 plus proches voisins de chaque pixel, ie les coins n'ont que 2 voisins et les bords que 3. Il est cependant possible de modifier le graphe et de considérer 8 voisins en changeant les paramètres du modèle.

Optimizer : Utilisation de Adam sans utiliser de scheduler. De plus les modèles avec SGD+ scheduler convergent très lentement la solution a donc été écartée.

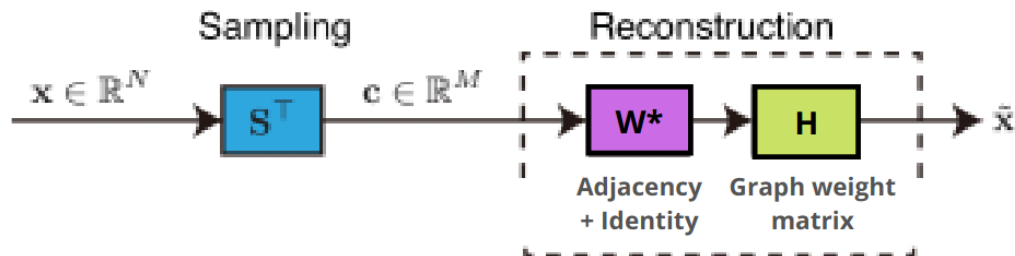
Criterion/Loss : Utilisation du MSELoss(). Pour tester l'accuracy du signal samplé et reconstruit, choix d'entraîner un MLP sur MNIST (accuracy sur la validation : 98.84%, accuracy sur le test : 97.11%) puis de tester ensuite l'accuracy du nouveau signal.

IV. Fonction train

- C'est une fonction récursive, qui permet de réaliser l'entraînement et le réentraînement.
- Pendant le réentraînement on regarde si la loss de la validation continue à baisser après 10 époques, dans le cas échéant le réentraînement est stoppé.
- La fonction train enregistre automatiquement les résultats dans results.csv
- Pour afficher les images des tenseurs et des matrices d'échantillonnage pendant l'entraînement il faut décommenter plot_results_epoch

V. Les modèles

Six modèles de Multi-Layer Perceptron ont été implémentés. Les 2 premiers ont une matrice d'échantillonnage générée aléatoirement. Les 4 suivants utilisent le pruning pour créer la matrice d'échantillonnage. Dans les deux derniers la signal échantillonné est ensuite multiplié le graphe pour diffuser le signal.



A. Les modèles aléatoires

- MLP_rd(dim_in, H, dim_out, k) :**
Ce premier modèle crée une matrice d'échantillonnage aléatoire qui sera utilisée pour sampler le signal durant le réentraînement.
- MPL_rd_pro(dim_in, H, dim_out, k) :**
Ce second modèle crée une matrice d'échantillonnage à partir d'une distribution. Cette distribution est donnée par la somme des intensités par pixel sur toutes les images. Cette matrice sera utilisée pour sampler le signal durant le réentraînement.

B. Les modèles utilisant le pruning

- Net0(dim_in, H, dim_out, k, neighbor = 4) :**
Ce troisième modèle contient une matrice d'échantillonnage **final_sampl** qui n'est utile qu'au réentraînement. Le pruning est appliqué au tenseur durant l'entraînement. A la fin de l'entraînement une matrice d'échantillonnage avec les nœuds dont les tenseurs n'ont pas été prunés est créée. Enfin le modèle est entraîné avec les signaux filtrés.
- Net1(dim_in, H, dim_out, k, neighbor = 4) :**
Le quatrième modèle contient une matrice d'échantillonnage différentiable **sampl_matrix** et une matrice d'échantillonnage non-différentiable **final_sampl**. Le pruning est appliqué sur **sampl_matrix** durant l'entraînement. A la fin de l'entraînement la matrice d'échantillonnage finale **final_sampl** est ajustée pour qu'elle contienne les nœuds de la matrice de **sampl_matrix** qui n'ont pas été prunés. Enfin le modèle est réentraîné avec les signaux filtrés.

C. Les modèles utilisant pruning et diffusion

a. **Net2(dim_in, H, dim_out, k, neighbor = 4) :**

Ce cinquième modèle a comme attributs : une matrice d'échantillonnage à pruner pour l'entraînement **sampl_matrix**, une matrice d'échantillonnage finale **final_sampl**, le graphe **W**, une matrice de poids différentiable **H**, une matrice de poids non-différentiable **fin_H**. Le pruning est effectué sur **sampl_matrix** durant l'entraînement. À la fin de l'entraînement on ajuste la matrice d'échantillonnage finale **final_sampl** pour qu'elle contienne les nœuds de la matrice de **sampl_matrix** qui n'ont pas été prunés. Puis maj de **fin_H**. Enfin le modèle est réentraîné avec les signaux filtrés.

b. **Net3(dim_in, H, dim_out, k, neighbor = 4) :**

Ce dernier modèle est construit comme le précédent mais ne possède pas de matrice de pondération du graphe (H et fin_H).

D. Table de récap

Multi-Layer perceptron	matrice d'échantillonnage aléatoire	Utilisation du pruning		Utilisation de la diffusion	
		Tenseur	Matrice d'échantillonnage	W* (Matrice d'adjacence + Identité)	Matrice de pondération de W*
MLP_rd	X				
MLP_rd_pro	X				
Net0		X			
Net1			X		
Net2			X	X	
Net3			X	X	X

VI. Résultats

A. Hyperparamètres

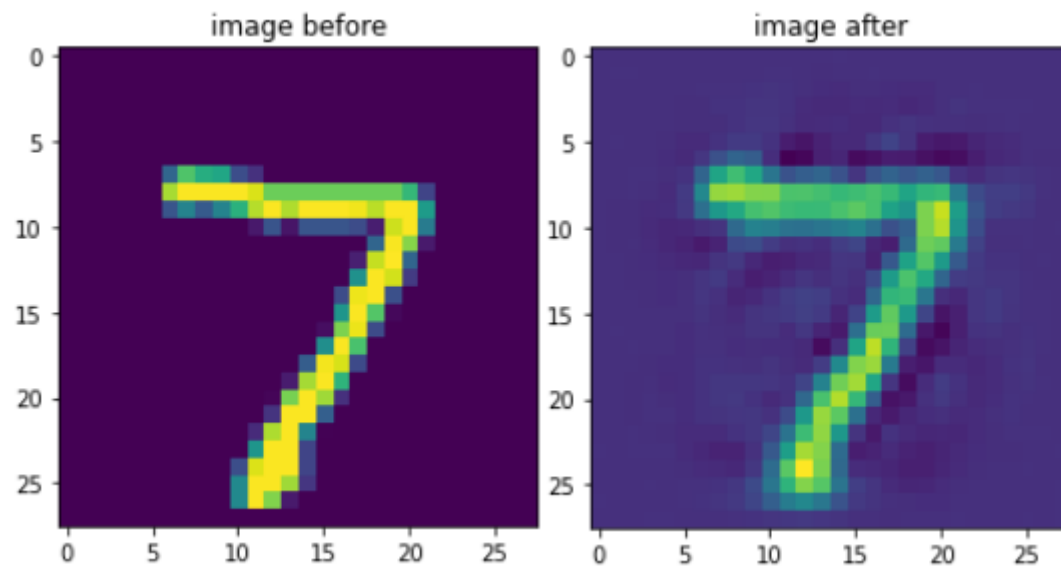
- a. Pour le ***select weight decay*** (swd) les paramètres à trouver son amin et amax. Ces derniers sont très variables en fonction du nombre de k pixels à pruner. J'ai choisi de garder ceux qui donnaient les meilleurs résultats pour k=100 pixels. Soit **amin = $10^{**}(-12)$ et amax = 100**.
- b. Le nombre d'époques choisies est **200**. Cette valeur offrait des performances correctes sans rendre l'entraînement trop long.

B. Table des résultats et analyse

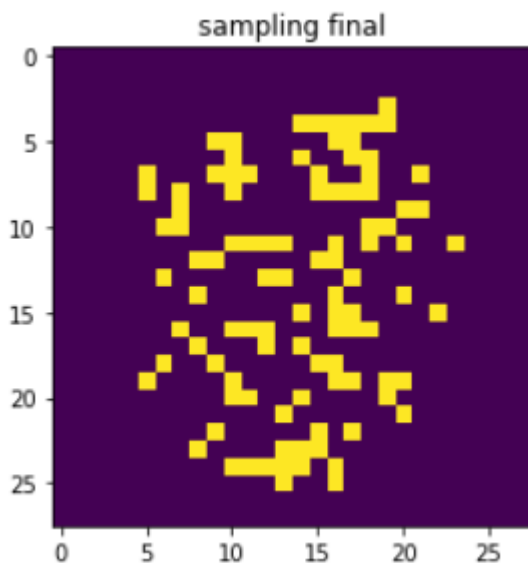
Les meilleures performances sont obtenues pour le modèle Net2 qui utilise le pruning sur une matrice d'échantillonnage différentiable et la diffusion en multipliant le signal échantillonné par un graphe pondéré. Ci dessous la table des résultats pour k= 100 et k=50. Chaque résultat correspond à la moyenne de 5 modèles. Ainsi il est très probable qu'en entraînant plus de modèle et en faisant de la cross validation les résultats soient un peu modifiés.

	MLP_rd		MLP_rd_pro		Net0		Net1		Net2		Net3	
	loss test (x100)	acc test	loss test (x100)	acc test	loss test (x100)	acc test	loss test (x100)	acc test	loss test (x100)	acc test	loss test (x100)	acc test
k = 100	2.56 ± 0.22	85.12± 2.55	2.18 ± 0.17	88.36± 1.44	1.80 ± 0.11	90.39± 1.19	1.55 ± 0.10	91.62± 0.6	1.37 ± 0.05	93.62± 0.33	1.61 ± 0.06	91.40± 0.48
k = 50	3.84 ± 0.25	62.00± 4.81	3.53 ± 0.31	70.05± 7.45	3.53 ± 0.31	77.04± 5.03	2.75 ± 0.22	77.56± 4.72	2.45 ± 0.11	83.75± 3.16	2.76 ± 0.14	78.85± 3.12

- Net1 qui applique le pruning à la matrice d'échantillonnage différentiable a de meilleures performances sur la loss et l'accuracy que Net0 qui applique le pruning sur le tenseur. Ce qui conforte dans le choix de garder cette architecture de base pour Net2.
- Pour Net2 les meilleurs résultats obtenus ont (loss, acc) : (0.0131, 94.6) et (0.0124, 94.11)
- Avec k =50 les écarts de performance se creusent.
- Les performances quand le pruning n'était pas appliqué (choix à la fin les poids de plus grande intensité) dans les réseaux Net0 à 3 étaient un peu moindre mais la loss chutent drastiquement lors du passage de l'entraînement au réentraînement. De plus, le réentraînement était plus long.
- Dans results.csv est répertorié l'époque de convergence de la matrice d'échantillonnage (*convergence_epoch*) ainsi que l'époque d'arrêt du réentraînement (*epoch*). Ces valeurs semblent plus dépendre de amax et amin.
- Visuellement le signal reconstruit par Net2 est le moins flou.
- Le temps de chaque entraînement + réentraînement est d'environ 15 minutes.
- Avant-après pour k = 100 :



- Matrice d'échantillonnage pour $k = 100$:



Ci dessous la table des résultats montrant les performances en fonction des k pixels à garder. Chaque résultat correspond à la moyenne pondérée de 3 à 5 modèles.

k = 200		k = 100		k = 50		k = 20		k = 10		k = 3	
loss test (x100)	acc test	loss test (x100)	acc test	loss test (x100)	acc test	loss test (x100)	acc test	loss test (x100)	acc test	loss test (x100)	acc test
0.76 ± 0.01	96.55 ± 0.13	1.37 ± 0.05	93.62 ± 0.33	2.45 ± 0.11	83.75 ± 3.16	4.03 ± 0.11	57.84 ± 4.84	5.11 ± 0.19	35.82 ± 4.57	6.18 ± 0.05	13.07 ± 2.10

- Les résultats diminuent en fonction du nombre de pixel gardés

- Quand $k = 200$ ce qui équivaut à prendre 1 pixel sur 4, le score est équivalent à celui obtenu avec les signaux non échantillonnés. Ce qui correspond à notre graphe où chaque pixel a 4 voisins.

VII. Conclusion et ouverture

Le modèle de pruning et reconstruction pour $k=100$ est fonctionnel mais ne permet pas une reconstruction complète du signal.

Il reste de nombreuses pistes à explorer pour continuer le sujet, notamment :

- Créer un graphe avec 8 voisins et évaluer les performances pour $k=100$. Puis comparer ces résultats avec ceux obtenus pour un graphe de 4 voisins. Pour évaluer l'importance de la précision et de la connectivité du graphe dans la reconstruction du signal.
- Évaluer quel est le meilleur échantillonnage pour $k = 3$ et comparer avec les résultats obtenus par les modèles pour vérifier leur proximité à la meilleure solution.
- Utiliser la cross-validation pour avoir des performances plus réalistes.
- Appliquer le pruning et la reconstruction avec un réseau de neurones convolutif plus adapté aux images.
- Changer la base de données pour étudier l'efficacité de la solution en fonction de la "smoothness" du signal